

Find Pairs With Given Sum in a Sorted Doubly Linked List

Personal DSA Notes

Given a sorted doubly linked list and a target sum, return all pairs whose values add up to that target. The sorted property is the key to the optimized solution. ■filecite■turn29file0■L10-L24■

1. Problem Statement

For example, with target 5, (1,4) and (2,3) are valid pairs. All valid pairs must be returned; their order does not matter. ■filecite■turn29file0■L26-L62■

2. Answer Data Structure

```
List<List<Integer>> result = new ArrayList<>();

// One pair:
result.add(Arrays.asList(left.val, right.val));
```

The outer list stores all pairs. Each inner list stores the two values of one pair.

3. Brute-Force Approach

Keep one pointer at the current node and another one ahead. Check pairs. Because the list is sorted, for a fixed first value, once the second value makes the sum exceed the target, later values will only be larger, so that inner scan can stop. ■filecite■turn29file0■L64-L108■

```
1 <-> 2 <-> 3 <-> 4 <-> 9
^      ^
t1     t2

Check 1+2, 1+3, 1+4 -> valid
1+9 > target -> stop this inner scan
```

This approach is approximately $O(n^2)$ time and $O(1)$ auxiliary space, excluding the output.

■filecite■turn29file0■L292-L328■

4. Optimized Idea: Two Pointers

Use the sorted property and the doubly linked list's backward `prev` pointer. Put `left` at the head (smallest value) and `right` at the tail (largest value). Then move them toward each other.

■filecite■turn29file0■L332-L360■

```
1 <-> 2 <-> 3 <-> 4 <-> 9
^              ^
left          right

left -> increases the value
right <- decreases the value
```

5. Pointer Rules

sum == target: store the pair, then move both pointers inward.

sum < target: the sum is too small, so move `left` forward.

sum > target: the sum is too large, so move `right` backward.

These rules follow directly from the sorted order. ■filecite■turn29file0■L364-L420■

```
target = 5
```

```
1 <-> 2 <-> 3 <-> 4 <-> 9
```

```

^           ^
L           R

1+9 = 10 > 5
move R backward

1 <-> 2 <-> 3 <-> 4 <-> 9
^           ^
L           R

1+4 = 5
store (1,4)
move both

^           ^
L           R
2 <-> 3

2+3 = 5
store (2,3)
move both -> crossed -> stop

```

6. Find the Tail

```

ListNode right = head;

while (right.next != null) {
    right = right.next;
}

```

This first traversal finds the tail so `right.prev` can be used during the two-pointer phase.

filecite turn29file0 L448-L460

7. Pseudocode

```

right = tail(head)
left = head
result = empty list

while left and right have not crossed:

    sum = left.val + right.val

    if sum == target:
        add [left.val, right.val]
        left = left.next
        right = right.prev

    else if sum < target:
        left = left.next

    else:
        right = right.prev

return result

```

8. Complete Java Code

```

class Solution {

    public List<List<Integer>> findPairsWithGivenSum(
        ListNode head, int target) {

        List<List<Integer>> result = new ArrayList<>();

        if (head == null) {
            return result;
        }
    }
}

```

```

// Find the tail so right can move backward.
ListNode right = head;
while (right.next != null) {
    right = right.next;
}

ListNode left = head;

// Stop when the two pointers meet or cross.
while (left != right && left.prev != right) {

    int sum = left.val + right.val;

    if (sum == target) {

        // Store this valid pair.
        result.add(Arrays.asList(left.val, right.val));

        left = left.next;
        right = right.prev;

    } else if (sum < target) {

        // Need a larger sum, so move left forward.
        left = left.next;

    } else {

        // Need a smaller sum, so move right backward.
        right = right.prev;

    }

}

return result;
}
}

```

Comments are intentionally limited to the lines that are useful for future revision: finding the tail, stopping condition, storing a pair, and the reason for each pointer movement.

9. Why the Pointers Move This Way

If the sum is too large, moving `left` forward would make the sum even larger, so it cannot help. Move `right` backward to reduce the sum. If the sum is too small, moving `right` backward would make it even smaller, so move `left` forward to increase it. ■filecite■turn29file0■L364-L396■

10. Why We Stop

When the pointers meet or cross, there are no new pairs left to check. The transcript describes stopping when the right value becomes smaller than the left value. ■filecite■turn29file0■L422-L436■

11. Dry Run

```

Input:
1 <-> 2 <-> 3 <-> 4 <-> 9
target = 5

L=1, R=9
10 > 5 -> R moves to 4

L=1, R=4
5 == 5 -> save [1,4]
L moves to 2, R moves to 3

L=2, R=3
5 == 5 -> save [2,3]
L and R move inward

```

Pointers cross -> stop

Result:

[[1,4], [2,3]]

12. Time Complexity

$O(n)$ overall. Finding the tail takes $O(n)$. After that, each two-pointer step moves either `left` forward or `right` backward, and neither pointer reverses direction. Therefore the two-pointer phase is $O(n)$. Total = $O(n) + O(n) = O(2n) = O(n)$. This matches the transcript's complexity reasoning. ■filecite■turn29file0■L550-L576■

13. Space Complexity

$O(1)$ auxiliary space. The algorithm uses only a constant number of pointers/variables. The returned result itself requires space proportional to the number of pairs, say $O(k)$. The transcript separates the algorithm's $O(1)$ space from output storage. ■filecite■turn29file0■L576-L600■

14. Complexity Summary

Time: $O(n)$
Auxiliary: $O(1)$
Output: $O(k)$, k = number of returned pairs

Remember:

The $O(1)$ space claim excludes the required output list.

15. Edge Cases

- Empty list → empty result.
- One node → no pair.
- Two nodes → one possible pair.
- No valid pairs → empty result.
- Multiple valid pairs → store every pair.
- Sum too small → left moves.
- Sum too large → right moves.
- Pointers meet/cross → stop.

16. Brute Force vs Optimized

Approach	Time	Auxiliary space	Key idea
Brute force	$\sim O(n^2)$	$O(1)$	Check many pairs
Two pointers	$O(n)$	$O(1)$	Use sorted order + prev pointer

17. Quick Revision

SORTED DLL + TARGET SUM

```
1. left = head
2. right = tail
3. while left and right have not crossed:
    sum = left.val + right.val

    sum == target:
        store pair
        left = left.next
        right = right.prev

    sum < target:
        left = left.next

    sum > target:
```

```
right = right.prev
```

4. Return result

Why it works:

Sorted order tells us which pointer to move.

Time: $O(n)$

Auxiliary space: $O(1)$

Output space: $O(k)$